

Lesson 4 — Security Middleware, Validation, and Password Hashing

Node.js/Express

Game Plan

- Warm-Up
- Protected Routes & Auth Middleware
- Data Validation with Joi
- Password Hashing
- Assignment Preview
- Wrap-Up

Warm-Up (5 min)

In chat or out loud:

1. What clicked for you in Assignment 3?
2. Can you think of a real website where you'd want some routes to be public and others private?

The Problem with Open Routes

Right now, anyone can call:

```
DELETE /api/tasks/5
```

...even without being logged in.

We need a way to protect certain routes. That's what **auth middleware** does.

Auth Middleware Pattern

```
// middleware/auth.js
module.exports = (req, res, next) => {
  if (!global.user_id) {
    return res.status(401).json({ message: "Unauthorized." });
  }
  next(); // logged in – pass to route handler
};
```

Notice: either `return res.json()` or `next()` — never both!

Applying Auth Middleware

You can protect an entire group of routes at once:

```
const authMiddleware = require("./middleware/auth");
const taskRouter = require("./routers/taskRoutes");

// All task routes require auth
app.use("/api/tasks", authMiddleware, taskRouter);
```

The login and register routes stay **unprotected** — users need to reach them first.

DRY: Don't Repeat Yourself

Without middleware, every route handler would have to check:

```
app.get("/api/tasks", (req, res) => {  
  if (!global.user_id) return res.status(401).json(...);  
  // ... actual logic  
});  
  
app.post("/api/tasks", (req, res) => {  
  if (!global.user_id) return res.status(401).json(...); // repeated!  
  // ... actual logic  
});
```

Middleware solves this — write the check once, apply it everywhere.

Quick Think (2 min)

If you had a blog with public posts but private drafts, which routes would you protect?

```
GET    /api/posts      (public)
GET    /api/posts/:id   (public)
POST   /api/posts      (???)
PATCH /api/posts/:id   (???)
DELETE /api/posts/:id   (???)
GET    /api/drafts      (???)
```

Data Validation

Before storing data, **validate** it.

Bad data leads to:

- Broken behavior
- Security vulnerabilities (injection attacks)
- Unhelpful error messages

We use **Joi** for validation in this project.

```
npm install joi
```

Joi Basics

```
const Joi = require("joi");

const schema = Joi.object({
  title: Joi.string().min(1).max(255).required(),
  isCompleted: Joi.boolean().default(false),
});

const { error, value } = schema.validate(req.body);
if (error) {
  return res.status(400).json({ message: error.details[0].message });
}
```

If validation passes, `value` contains the cleaned data.

Joi Can Transform Too

```
const userSchema = Joi.object({  
  name: Joi.string().trim().required(),  
  email: Joi.string().email().lowercase().required(),  
  password: Joi.string().min(8).required(),  
});
```

- `.trim()` — removes leading/trailing whitespace
- `.lowercase()` — normalizes emails
- `.email()` — validates format

Validation is your **first line of defense** against bad input.

Predict This

```
const schema = Joi.object({  
  password: Joi.string().min(8).required(),  
});  
  
const { error } = schema.validate({ password: "abc" });
```

What does `error` contain? What HTTP status should you return?

Password Storage

Never store plain-text passwords.

If your database is compromised, attackers get every password.

Instead: store a **cryptographic hash**.

- A hash is a one-way transformation
- You can verify a password by hashing it and comparing
- Each password gets a unique **salt** to prevent rainbow table attacks

bcrypt / scrypt

Node's built-in `crypto` module has `scryptSync` :

```
const crypto = require("crypto");

// Hashing (on registration)
const salt = crypto.randomBytes(16).toString("hex");
const hash = crypto.scryptSync(password, salt, 64).toString("hex");
const storedHash = `${salt}:${hash}`;

// Verifying (on login)
const [savedSalt, savedHash] = storedHash.split(":");
const hashToCheck = crypto.scryptSync(inputPassword, savedSalt, 64).toString("hex");
const match = hashToCheck === savedHash;
```

Security Rules

1. Use a well-known, public hashing algorithm — **never invent your own**
2. Always use a unique salt per password
3. Never store the plain-text password, even temporarily
4. Never store credit card numbers, SSNs, etc. without proper compliance

"Never invent your own cryptography." — every security expert ever

We Do — Spot the Bug

Which of these is safer and why?

```
// Option A
global.users.push({ ...req.body });

// Option B
const { name, email, password } = req.body;
const { error, value } = userSchema.validate({ name, email, password });
if (error) return res.status(400).json({ message: error.details[0].message });
global.users.push(value);
```

We Do — Design a Validation Schema

For this task shape:

```
{  
  title: "Buy groceries",  
  isCompleted: false  
}
```

What Joi rules would you write?

- `title` — required? max length? what type?
- `isCompleted` — required? default value?

You Do (5 min)

Write a Joi schema for a task `update` operation:

- `title` should be optional (you might only update one field)
- `isCompleted` should be optional but must be boolean if present
- Reject any unknown fields

Hint: `Joi.object({ ... }).options({ allowUnknown: false })`

Assignment Preview

This week's assignment is the biggest one so far:

1. Build all 5 task routes: create, index, show, update, deleteTask
2. Add auth middleware in front of all task routes
3. Add Joi validation for user and task create/update
4. Hash passwords on register (using `crypto.encryptSync`)
5. Verify hashed password on login

The assignment is labeled "a lot of work" — give yourself extra time!

Assignment: Task ID Pattern

```
const taskCounter = (() => {  
  let lastTaskNumber = 0;  
  return () => {  
    lastTaskNumber += 1;  
    return lastTaskNumber;  
  };  
})();
```

This is a **closure** — a function that remembers its own state.

Each call to `taskCounter()` returns the next unique ID.

Wrap-Up

In chat:

1. Why do we use middleware for auth instead of putting the check in every route?
2. Why should we never store a plain-text password?
3. What's the difference between a validation error (400) and an unauthorized error (401)?

Confidence Check

On a scale of 1–5:

How confident are you about building protected routes this week?

Resources

- <https://joi.dev/api/>
- <https://nodejs.org/api/crypto.html>
- Ask questions in Slack

Closing

This week:

Auth middleware, Joi validation, and proper password storage.

Next week:

We switch from in-memory arrays to a real database — SQL and PostgreSQL.

See you then!